VULNERABILITY ASSESSMENT & PENETRATION TESTING REPORT

Vault Notes
A Deliberately Vulnerable Full-Stack Notes Application

Prepared By:	Raghvendra Tripathi
Target Application:	Vault Notes (github.com/Raghavtripathii/vault-notes-vapt)
Assessment Type:	White-box Web Application Penetration Test
Report Date:	July 10, 2026
Classification:	Educational / Portfolio Assessment

Executive Summary

This report documents a self-conducted penetration test against Vault Notes, a full-stack notes-taking web application built specifically for this assessment. The application was first developed as a fully functional, securely-coded system (authentication, protected note CRUD operations, and a React frontend). Five distinct vulnerabilities were then deliberately introduced, one at a time, and subsequently discovered and exploited using manual testing techniques and Postman, without automated scanning tools.

The purpose of this exercise was to build hands-on, verifiable experience across the full vulnerability lifecycle: secure development, vulnerability introduction, manual exploitation, and professional-grade reporting — the same workflow expected of an entry-level Web Application Penetration Tester / VAPT Analyst.

Five findings were identified, spanning Critical and High severity, covering authentication bypass, broken access control, client-side script injection, insecure credential storage, and hardcoded secret exposure. A summary is provided below, with full technical detail in the Findings section.

Findings Summary

#	Finding	Severity	OWASP Category
1	SQL Injection – Authentication Bypass	**Critical**	A03:2021 Injection
2	IDOR – Unauthorized Note Access	**High**	A01:2021 Broken Access Control
3	Stored XSS in Note Content	**High**	A03:2021 Injection
4	Plaintext Password Storage	**Critical**	A02:2021 Cryptographic Failures
5	Hardcoded Secret Exposed in Frontend	**Critical**	A02:2021 Cryptographic Failures

Scope

The assessment covered the Vault Notes backend (Node.js / Express / SQLite, JWT authentication) and frontend (React / TypeScript), running in a local development environment. Testing was performed with full knowledge of the source code (white-box) and full authorization, as this is a self-built application created solely for this assessment.

Methodology

Testing followed a manual, OWASP Top 10-aligned approach. Each vulnerability class was reasoned about at the code level, reproduced with a proof-of-concept request via Postman, and verified end-to-end before being documented with severity, impact, and remediation guidance.

Finding #1: SQL Injection – Authentication Bypass

Severity:	**Critical**
Location:	POST /login
Vulnerability Class:	OWASP Top 10 — A03:2021 Injection

Description

The login endpoint builds its SQL query by directly concatenating user-supplied **username** and **password** values into the query string, instead of using parameterized queries. This allows an attacker to inject SQL syntax that alters the query's logic.

Proof of Concept

Request body sent to **POST /login**:

```
{
"username": "' OR '1'='1' --",
"password": "anything"
}
```

This transforms the intended query:

```
SELECT * FROM users WHERE username = '[input]' AND password = '[input]'
```

into:

```
SELECT * FROM users WHERE username = '' OR '1'='1' --' AND password = 'anything'
```

The **--** comments out the rest of the query, and **'1'='1'** is always true, matching the first row in the users table regardless of actual credentials. The server responded with a valid JWT token, granting full authenticated access without knowing any real password.

Impact

An attacker can log in as any user (typically the first account in the database, often an admin in real systems) without knowing their credentials, gaining full access to that account's private notes.

Remediation

Use parameterized queries (prepared statements) for all database operations:

```
const query = `SELECT * FROM users WHERE username = ? AND password = ?`;
db.get(query, [username, password], ...);
```

This was in fact how the endpoint was originally implemented before this vulnerability was intentionally introduced for this assessment.

Finding #2: IDOR – Unauthorized Note Access

Severity:	**High**
Location:	GET /notes/:id
Vulnerability Class:	OWASP Top 10 — A01:2021 Broken Access Control

Description

The endpoint for retrieving a single note by ID checks that the requester has a valid authentication token, but does not verify that the note actually belongs to the authenticated user. Note IDs are sequential integers, so any logged-in user can read any other user's private notes simply by changing the ID in the URL.

Proof of Concept

1. Created account **victim1**, logged in, and created a note via POST /notes with body:

```
{ "title": "Victim Secret", "content": "My bank PIN is 4821" }
```

Server responded with **noteId: 3**.

2. Created a separate, unrelated account **attacker1** and logged in, obtaining a valid token for this different account.

3. Sent **GET /notes/3** using the attacker1 token (not victim1's).

4. Server responded with 200 OK and returned victim1's full private note content:

```
{
"id": 3,
"user_id": 3,
"title": "Victim Secret",
"content": "My bank PIN is 4821"
}
```

Impact

Any authenticated user can enumerate note IDs (1, 2, 3, 4...) to read the private note content of every other user on the platform, regardless of ownership. In a real deployment, this could expose sensitive personal information at scale with minimal effort.

Remediation

Always scope database lookups to the authenticated user's own ID:

```
const query = `SELECT * FROM notes WHERE id = ? AND user_id = ?`;
db.get(query, [noteId, req.user.userId], ...);
```

Finding #3: Stored Cross-Site Scripting (XSS) in Note Content

Severity:	**High**
Location:	Frontend rendering of note content (NotesDashboard.tsx)
Vulnerability Class:	OWASP Top 10 — A03:2021 Injection (XSS)

Description

Note content is rendered directly into the page using React's dangerouslySetInnerHTML, which injects raw HTML without escaping it. Since note content is fully attacker-controlled, this allows stored JavaScript to execute in the browser of anyone who views the note.

Proof of Concept

Attempt 1 (blocked): a note was created with the content:

```
<script>alert('XSS: this note ran JavaScript in your browser')</script>
```

This did not execute. Modern browsers silently strip <script> tags inserted via innerHTML as a legacy protective measure, so this payload, while confirming raw HTML was being injected, did not prove active code execution on its own.

Attempt 2 (successful): a note was created with the content:

```
<img src="x" onerror="alert('XSS: this note ran JavaScript in your browser')">
```

Since the image source "x" is invalid, the browser fires the onerror event handler, which — unlike <script> tags — executes normally when injected via innerHTML. Upon viewing the notes list, a JavaScript alert popup fired immediately, confirming genuine script execution triggered purely by note content.

Impact

Any user can craft a note containing malicious JavaScript. If that note were ever viewed by another user (e.g. in a shared or admin-viewable context), the attacker's script would run in that victim's browser session — capable of stealing their auth token from memory, performing actions on their behalf, or redirecting them to a malicious site.

Remediation

Never use dangerouslySetInnerHTML for user-supplied content. Render as plain text instead:

```
<p>{note.content}</p>
```

If rich text formatting is genuinely required, use a dedicated sanitization library (e.g. DOMPurify) to strip dangerous tags/attributes before rendering.

Finding #4: Plaintext Password Storage

Severity:	**Critical**
Location:	users table (SQLite database)
Vulnerability Class:	OWASP Top 10 — A02:2021 Cryptographic Failures

Description

User passwords are stored in the database exactly as entered at signup, with no hashing or encryption applied. The login route also compares passwords directly as plain strings. Anyone with access to the database file can read every user's real password in cleartext.

Proof of Concept

Querying the users table directly:

```
db.all("SELECT id, username, password FROM users", (err, rows) => console.log(rows));
```

returned every user's password as plain, human-readable text:

```
[
{ id: 1, username: 'raghav', password: 'test2005' },
{ id: 2, username: 'Mahi Tripathi', password: '2004' },
{ id: 3, username: 'victim1', password: 'victimpass123' },
{ id: 4, username: 'attacker1', password: 'attackerpass123' }
]
```

Impact

Since many people reuse passwords across services, a leaked database here could let an attacker log into a victim's email, banking, or other accounts elsewhere. This is one of the most damaging and common real-world vulnerabilities, frequently responsible for large-scale credential-stuffing attacks after data breaches.

Remediation

Hash passwords using a slow, purpose-built algorithm such as bcrypt:

```
const bcrypt = require("bcrypt");

// at signup:
const hashedPassword = await bcrypt.hash(password, 10);

// at login:
const match = await bcrypt.compare(password, user.password);
```

Finding #5: Hardcoded Secret Exposed in Frontend Code

Severity:	**Critical**
Location:	frontend/src/api.ts
Vulnerability Class:	OWASP Top 10 — A02:2021 Cryptographic Failures / CWE-798

Description

The backend's JWT signing secret was hardcoded directly into frontend source code. Frontend code is bundled and shipped to every visitor's browser, meaning any secret placed there is effectively public the moment the site is deployed. Since this project's repository is public on GitHub, the secret is also permanently visible in the commit history.

Proof of Concept

1. Opened the live frontend application in Chrome DevTools → Sources tab, and located src/api.ts served in plain, unminified form, containing the literal JWT secret string.

2. Navigated to the public GitHub repository and confirmed the same secret is visible directly in the file browser at frontend/src/api.ts, accessible to any visitor without authentication.

Impact

Anyone with this secret can forge their own valid JWT tokens for any user ID of their choosing — completely bypassing the login process entirely, without needing a password, a SQL injection, or any other vulnerability. This is effectively a master key to the entire application's authentication system, and is arguably the most severe finding in this assessment since it undermines authentication at its root, regardless of any other fixes applied elsewhere.

Remediation

• Never place backend secrets (API keys, signing secrets, database credentials) in frontend code under any circumstances.
• Store secrets only in backend environment variables (.env), excluded from version control via .gitignore.
• If a secret is ever accidentally committed to a public repository, treat it as compromised immediately: rotate/regenerate the secret, and consider the git history itself compromised unless properly scrubbed.

Conclusion

This assessment identified five vulnerabilities across authentication, access control, client-side rendering, credential storage, and secrets management — a representative cross-section of common real-world web application security failures. Each finding was independently reproduced with a working proof-of-concept and documented with remediation guidance aligned to secure coding practice.

All identified issues stem from well-understood root causes (missing input parameterization, missing ownership checks, unsafe HTML rendering, absent hashing, and improper secret placement) and are correctable using standard, well-documented fixes, several of which were already present in this application's initial secure implementation prior to the intentional introduction of these vulnerabilities for this exercise.

Full source code, commit history documenting the secure-to-vulnerable transformation, and this report are available at: github.com/Raghavtripathii/vault-notes-vapt